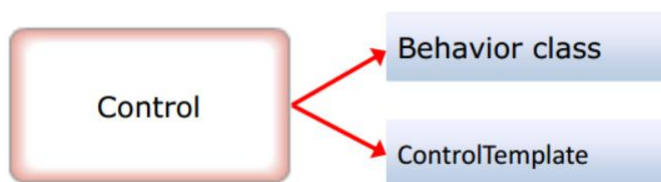


Templates

Controls, Styles und Templates

- Alle WPF-Elemente können nach belieben gestaltet werden
 - Strikte Trennung von Funktionalität und Aussehen
 - Bei Änderung des Aussehens, Funktionalität gleich
 - ähnlich wie in CSS



Controls bestehen aus zwei unterschiedlichen, aber miteinander kooperierenden Elementen:

Die Behavior-Class (der ‚Code-Behind‘) ist für das Verhalten des Controls zuständig. Sie enthält die Logiken, die auf das Control angewendet werden sollen. Dies können beispielsweise Event-Handler, DependencyProperties, Validierungslogik oder andere Funktionen sein. Die Behavior-Class ist festgelegt und nicht veränderbar. D.h. ein Button wird, unabhängig von seinem Aussehen, funktional immer ein Button sein.

Das ControlTemplate hingegen ist für das Aussehen und die Darstellung des Controls verantwortlich. Es definiert die visuelle Struktur (den ‚VisualTree‘) und das Design des Controls. Das ControlTemplate kann verschiedene XAML-Elemente wie Container, Styles, Trigger und andere Controls enthalten, um das gewünschte Erscheinungsbild des Controls zu erstellen. Durch die Trennung des ControlTemplates vom Verhalten des Controls wird die Flexibilität erhöht, da das Aussehen des Controls leicht geändert werden kann, ohne die Logik des Behaviors zu beeinflussen. Ein Designer kann jederzeit den VisualTree eines Controls durch Austausch des ControlTemplates verändern.

Templates

- Beschreibt Struktur von Controls oder Daten
- Zuweisung eines neuen Visual Tree

Arten:

- **ControlTemplate**
 - Beschreibt Struktur von Controls
- **DataTemplate**
 - Definiert Anzeige für Daten, z.B. DataTemplate für ListBoxItems
- **ItemsPanelTemplate**
 - Definiert Layouting eines ItemControl, z.B. ListBox

Es gibt verschiedene Arten von Templates, welche jeweils den VisualTree von unterschiedlichen WPF-Aspekten manipulieren können:

1. **ControlTemplate:** Ein ControlTemplate ist ein XAML-Markup, das das Aussehen und das visuelle Verhalten eines WPF-Steuerelements definiert. Es enthält die Struktur und das Design des Steuerelements, einschließlich seiner visuellen Elemente, wie z. B. Layouts, Stile, Trigger und Bindungen. Mit einem ControlTemplate kann das Erscheinungsbild eines Steuerelements vollständig angepasst bzw. ersetzt werden.
2. **DataTemplate:** Ein DataTemplate ist ein XAML-Markup, das die Darstellung von Datenobjekten in einem WPF-Steuerelement definiert, das Daten bindet (z.B. ListBox, ComboBox, etc.). Es enthält die visuelle Darstellung der Daten, z. B. die Anordnung der Steuerelemente und die Formatierung der Datenwerte. Ein DataTemplate definiert dabei die Darstellung eines Datensatzes und wird für weitere Datensätze repliziert.
3. **ItemsPanelTemplate:** Ein ItemsPanelTemplate ist ein XAML-Markup, das das Layout und die Anordnung von Elementen in einem ItemsControl definiert, das eine Liste von Elementen darstellt. Es definiert das Panel, das verwendet wird, um die Elemente anzuordnen, z. B. ein StackPanel, ein UniformGrid oder ein VirtualizingStackPanel. Mit einem ItemsPanelTemplate kann das Layout der Elemente in einem ItemsControl geändert werden, ohne das DataTemplate oder das ControlTemplate des Elements zu ändern.

ControlTemplate

- ControlTemplates ersetzen den VisualTree eines Controls
 - -> individuellere Anpassung als Styles
- Können jedes Control, jeden Container und die meisten WPF-Mechanismen (z.B. Bindungen) beinhalten
- TemplateBinding-Bindung zur relativen Anbindung an Properties des Original-Objekts
- Werden häufig durch Styles verteilt

```
<ControlTemplate TargetType="Button">
  <Grid>
    <Border BorderThickness="2" BorderBrush="Orange" CornerRadius="10">
      <Rectangle Fill="{TemplateBinding Background}" RadiusX="10" RadiusY="10"/>
    </Border>
    <ContentPresenter VerticalAlignment="Center" HorizontalAlignment="Center"/>
  </Grid>
</ControlTemplate>
```



Hallo Welt

Ein ControlTemplate ist ein Objekt, welches das Aussehen und das visuelle Verhalten eines WPF-Steuerelements definiert. Es ermöglicht die vollständige Anpassung des Erscheinungsbilds eines Steuerelements, indem es die visuellen Elemente, das Layout, die Styles, Trigger und Bindungen festlegt.

Ein ControlTemplate besteht aus einer Hierarchie von XAML-Elementen, die das visuelle Design des Steuerelements repräsentieren und den ursprünglichen VisualTree ersetzen. Es kann verwendet werden, um das Standardaussehen eines Steuerelements zu ändern oder ein komplett neues Design zu erstellen. Dazu wird es in die ‚Template‘-Eigenschaft des jeweiligen Steuerelements gelegt. Häufig werden ControlTemplates über Styles verteilt, um die Vorteile dieser (globale Verteilung, neue Standardwerte, Trigger) zur Verfügung zu haben.

Zur Anbindung einer Eigenschaft innerhalb des Templates an eine Eigenschaft des verwendeten Controls wird die spezielle relative Bindung ‚TemplateBinding‘ verwendet. So ermöglicht man dem Designer Eigenschaftsveränderungen im Original-Control auf den neuen VisualTree zu übertragen.

DataTemplate

- Beschreibt den VisualTree für einzelne Datensätze in ItemControls
- Wird für jeden Datensatz der Datenquelle repliziert
- DataContext innerhalb des DataTemplates sind immer die jeweiligen Datensätze

```
<ListBox.ItemTemplate>
  <DataTemplate DataType="{x:Type local:Person}">
    <Grid Height="60">
      ...
      <TextBlock Margin="5,0,0,0" Grid.Row="0" Grid.Column="1" Text="{Binding PersonName}"/>
      <TextBlock Margin="5,0,0,0" Grid.Row="1" Grid.Column="1" Text="{Binding Address}"/>
      <TextBlock Margin="5,0,0,0" Grid.Row="2" Grid.Column="1" Text="{Binding PhoneNumber}"/>
      ...
    </Grid>
  </DataTemplate>
</ListBox.ItemTemplate>
```

Ein DataTemplate ist ein Objekt, welches die Darstellung von Datenobjekten in WPF definiert. Es ermöglicht die individuelle Gestaltung des VisualTrees von Daten, indem es das Layout und das Erscheinungsbild der Datenobjekte festlegt.

Es wird in den meisten Fällen verwendet, um die visuelle Darstellung eines Datenobjekts zu definieren, das in einem ItemControl wie z.B. ListBox, ListView oder ComboBox angezeigt wird, kann aber auch (wenn als globale Ressource gesetzt) die Anzeige z.B. in ContentControls wie Labels definieren. Es beschreibt, wie die Daten in visuelle Elemente umgewandelt werden sollen, um sie dem Benutzer anzuzeigen.

Ein DataTemplate enthält XAML-Elemente, die die visuelle Struktur definieren, wie z.B. Textblocks, Bilder oder andere Steuerelemente, die die Daten repräsentieren sollen. Es kann auch Triggers, Bindungen und andere XAML-Funktionen enthalten, um die Darstellung der Daten basierend auf bestimmten Bedingungen oder Eigenschaften anzupassen.

Innerhalb von DataTemplates ist der DataContext automatisch der jeweilige Datensatz, was ein direktes DataBinding an die einzelnen Eigenschaften der Datenobjekte erlaubt.